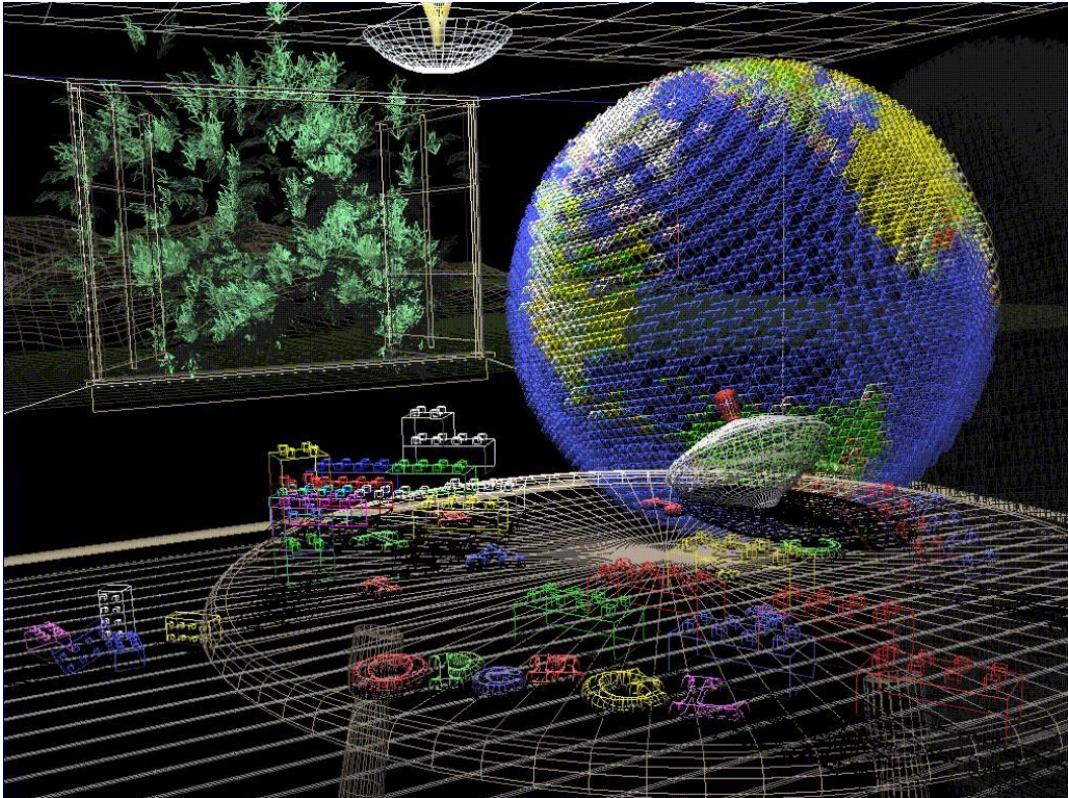


Practical Assignment CG

2024/25

Phase 1



Team members:

- Diego Sánchez Vargas
- Lucas González Puente

02/03/2025



Index

Introduction

Development

Architecture Diagram

Algorithms

Relevant Equations

Results

Conclusions

References

Appendices

Introduction

This report documents the development of a mini 3D engine and a model generator for the Computer Graphics course (2024/25). The project consists of two main components: the Generator and the Engine.

The Generator creates 3D model files for graphical primitives (planes, boxes, spheres, and cones) based on user-defined parameters. These models are stored in .3d files, which the Engine reads and renders. The Engine uses XML configuration files to set up the scene, camera, and models, and leverages OpenGL and GLUT for rendering and window management.

This report outlines the implementation of both components, including design decisions, data structures, and challenges faced. It also evaluates the results, comparing the rendered output with the expected visualizations. The project was developed in C++ using TinyXML2 for XML parsing and OpenGL/GLUT for rendering.

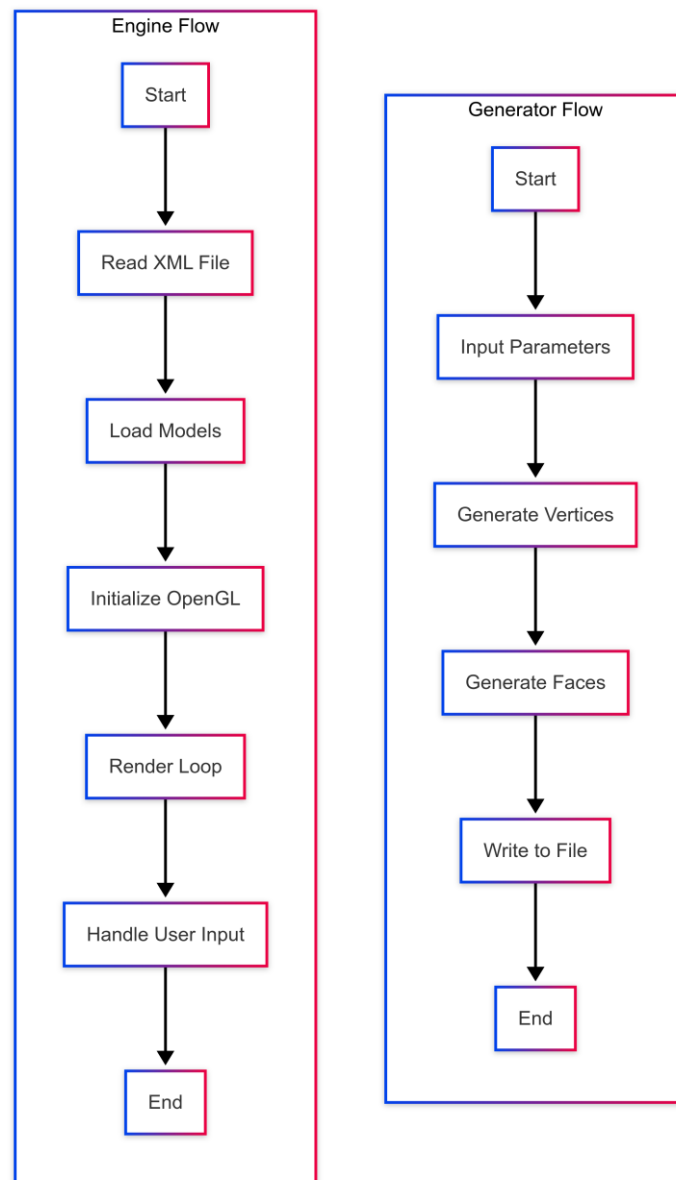
Technologies used:

- C++: Primary language.
- OpenGL/GLUT: Rendering and window management.
- TinyXML2: XML parsing.
- CMake: Build system for managing the project.

Development

Architecture Diagram

- Workflow Description:
 - Generator: Takes input parameters (primitive type, dimensions, divisions, etc.) and generates .3d files containing vertices and faces of the models.
 - Engine: Reads the XML configuration file, loads models from .3d files, and renders them in a window using OpenGL.
- Component Interaction:
 - The Generator and Engine are independent but work together through .3d files and the XML configuration file.
- Flow Diagram:



Data Structures

1. Vertex:
 - Represents a 3D point with x, y, and z coordinates.
 - Used to store the vertices of models.
2. Model:
 - Contains:
 - A vector<float> for vertices (stored as x, y, z triplets).
 - A vector<int> for faces (stored as triplets of vertex indices).
 - Used to store the geometry of 3D models.
3. Camera:
 - Contains:
 - Position (posX, posY, posZ).

- Look-at point (lookAtX, lookAtY, lookAtZ).
 - Up vector (upX, upY, upZ).
 - Projection settings (fov, near, far).
 - Used to configure the view and projection matrices in OpenGL.
4. Window:
- Contains:
 - Width and height of the rendering window.
 - Used to set up the OpenGL viewport and aspect ratio.
5. Model Colors: (Only to be able to tell apart the figures)
- A vector<vector<float>> containing RGB color values for models.
 - Ensures that each model is rendered with a distinct color.
6. Spherical Coordinates: (EXTRA)
- Used to control the camera's position dynamically:
 - theta: Horizontal angle (rotation around the Y-axis).
 - phi: Vertical angle (rotation above/below the XZ plane).
 - radius: Distance from the origin.

Algorithms

- Model Generation Algorithms:

1. Plane Generation:

Divides a square in the XZ plane into a grid of vertices based on the specified size and number of divisions. Generates triangles by connecting vertices in a grid pattern.

2. Box Generation:

Creates six faces (front, back, left, right, top, bottom) of a cube, each subdivided into a grid of vertices. Generates triangles for each face by connecting vertices in a grid pattern.

3. Sphere Generation:

Uses spherical coordinates to generate vertices distributed across the surface of a sphere. Connects vertices to form triangles using slices (horizontal divisions) and stacks (vertical divisions).

4. Cone Generation:

Generates vertices along the height of the cone, with the radius decreasing linearly from the base to the apex. Connects vertices to form triangles, including the base and the sides of the cone.

- Rendering Algorithm:

- Model Rendering: Iterates through the vertices and faces of each model, using OpenGL's `glBegin(GL_TRIANGLES)` to render each triangle. Applies a color to each model using `glColor3f()`.
- Camera Setup: Uses spherical coordinates (theta, phi, radius) to calculate the camera's position dynamically. Configures the camera using `gluPerspective()` for projection and `gluLookAt()` for the view matrix.
- XML Parsing Algorithm:

Uses TinyXML2 to parse the XML configuration file. Extracts camera settings (position, look-at point, up vector, projection) and model file paths.
- (EXTRA)Keyboard Input Handling:

Allows the user to rotate the camera around the scene using the W, A, S, and D keys. Updates the camera's spherical coordinates (theta, phi) and re-renders the scene.

Relevant Equations

1. Vertex Generation:

○ Plane:

$$x = -\frac{size}{2} + i \cdot \left(\frac{size}{divisions}\right), \quad z = -\frac{size}{2} + j \cdot \left(\frac{size}{divisions}\right)$$

○ Sphere:

$$x = r \cdot \sin(\phi) \cdot \cos(\theta), \quad y = r \cdot \cos(\phi), \quad z = r \cdot \sin(\phi) \cdot \sin(\theta)$$

Where:

- ϕ is the vertical angle.
- θ is the horizontal angle.

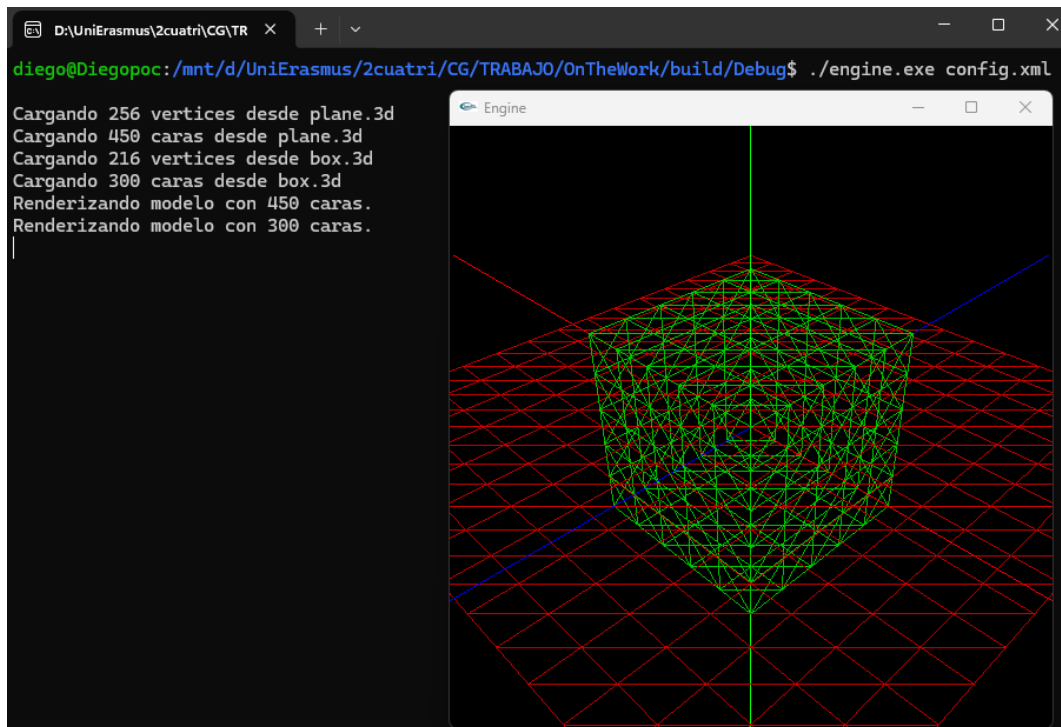
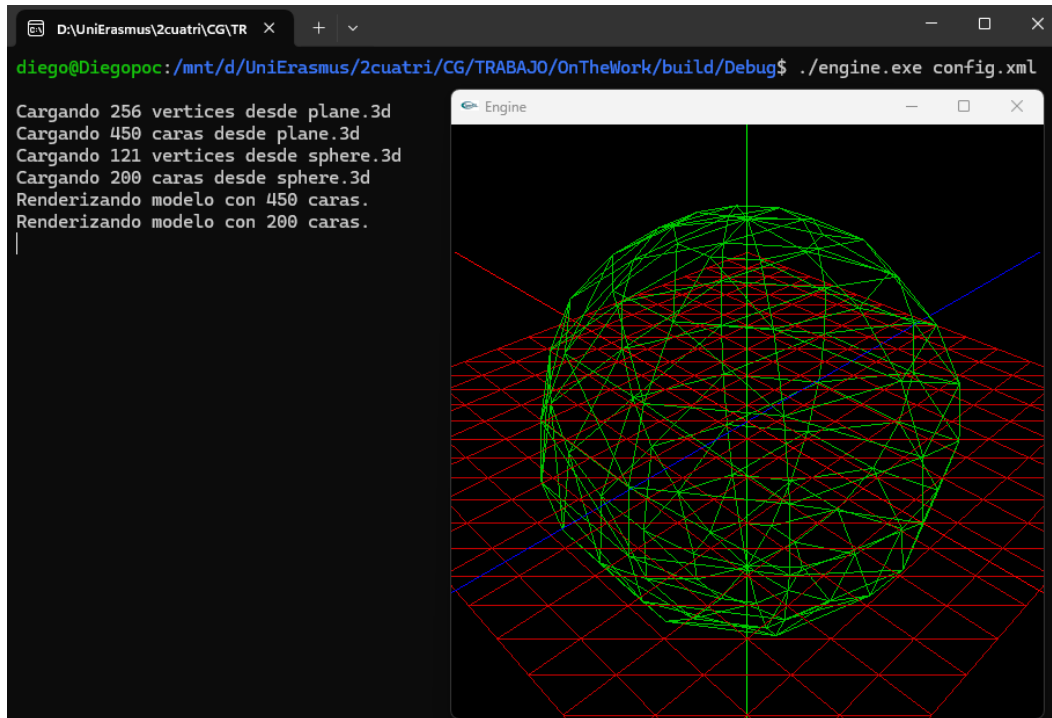
○ Cone:

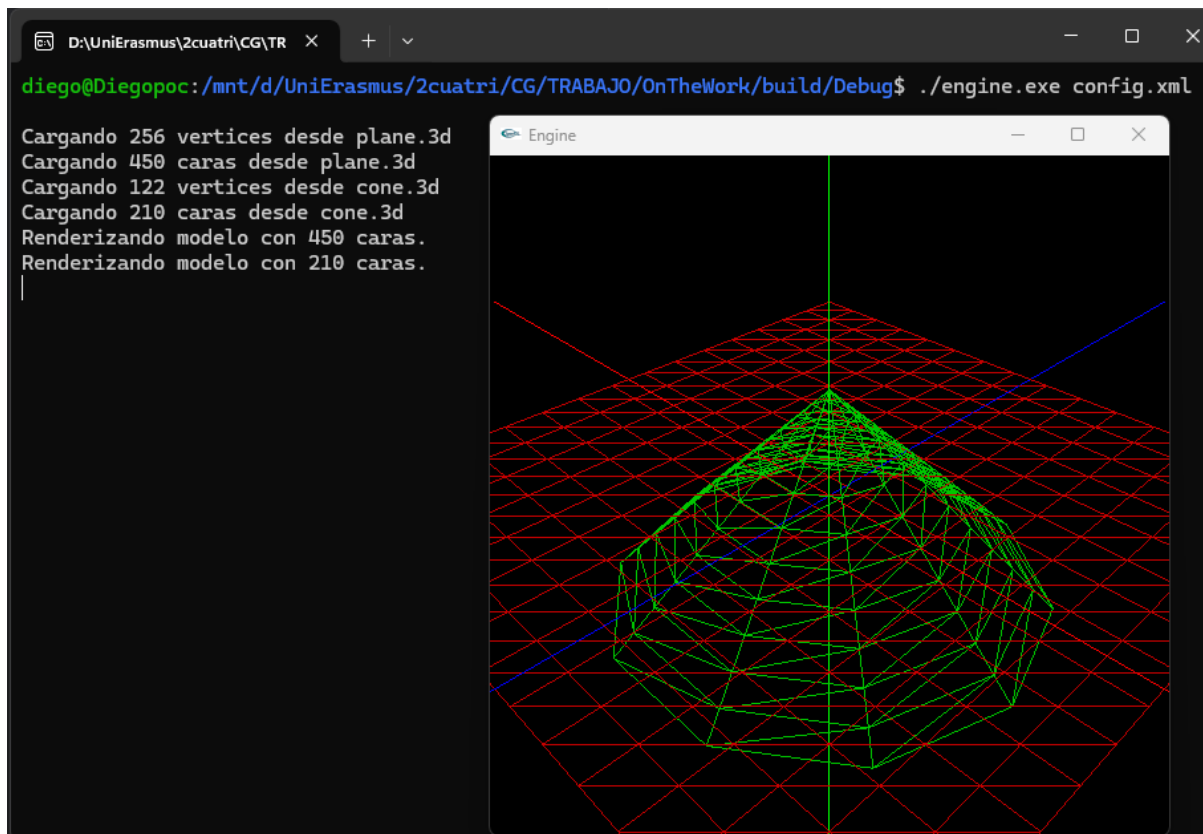
$$x = r \cdot \left(1 - \frac{i}{stacks}\right) \cdot \cos(\theta), \quad z = r \cdot \left(1 - \frac{i}{stacks}\right) \cdot \sin(\theta)$$

Where:

- r is the base radius.
- i is the current stack index.

Results





Conclusions

This project has given us valuable insights into the fundamentals of 3D graphics, including model generation, scene management, and rendering. The Phase 1 starts a good project where more advanced features and optimizations can be implemented. The experience that I gained from this project will be really important in tackling more complex challenges in computer graphics.

So far everything was more or less easy to carry on with.

- Future Improvements
 - Lighting and Shadows:
 - Implement basic lighting to improve depth perception and realism in the rendered models.
 - Textures:
 - Add support for textures to make models more visually appealing.
 - Enhanced User Interaction:
 - Allow zoom, pan, and free camera movement for a more immersive experience.
 - Optimization:
 - Optimize the code to handle larger and more complex scenes efficiently.
- Advanced Features:
 - Introduce additional primitives (e.g., torus, cylinder) and support for more complex scene graphs.

References

Everything used to develop the application came from the practical lessons files and ChatGPT to merge the ideas in place.

Anex

Cmake usage:

